

3. Voraussetzungen

4. Quelltext

- Kommentare
- Code

5. Pakete, Klassen, Variablen

6. Methoden

Kommentare

Kommentare

Formen:

- **Zeilenkommentare** `// ...` für einzeilige Kommentare, z.B. Sinn von Variable oder Anweisung, Zwischenstand, ...
- **Blockkommentare** `/* ... */` für mehrzeilige Kommentare, vor allem zu Klassen, Methoden, komplexeren Codeabschnitten

Hinweis: Zeichenfolge `*/` in Blockkommentar nicht möglich

Kommentare **dokumentieren** Quellcode:

- **allgemein:** Urheberschaft, Voraussetzungen, Kontext
- für **Anwender:** Zweck und Verwendung
- für **Entwickler:** Struktur und Vorgehen
- auch für **Autor:** Verdeutlichung, Erinnerung

Hinweis: Kommentare **knapp, präzise, vollständig** halten

Dokumentationskommentare

Entwickler hat Quellcode mit Kommentaren, Anwender nur Bytecode
⇒ Code-Kommentare **ungeeignet für Anwender**

javadoc-Kommentar `/** ... */` (v.a. Klassen / Methoden),
zur **Dokumentation extern relevanter Eigenschaften**
(kein Bezug auf interne Struktur und Vorgehensweise)

Tool **javadoc** generiert daraus **Benutzerhandbuch**

Konvention: javadoc-Kommentare in folgender Form:

```
/**  
 * ... (Text)  
 */
```

Tags der Form **@*merkmal*** im Kommentar für bestimmte Merkmale

Dokumentationskommentare: Beispiel

```
1  /**
2   * Geometrische Berechnungen.
3   * @author A. Priesnitz - andreas.priesnitz@haw-hamburg.de
4   * @version 1.0
5   * @since 2015-10-05
6   */
7  public class GeometrieKommentiert {
8      /**
9       * Berechnet Umfang eines Rechtecks gegebener Ausdehnung.
10      * Wert 0 fuer Laengen ist erlaubt.
11      * @param breite Breite des Rechtecks (selbe Einheit)
12      * @param hoehe Hoehe des Rechtecks (selbe Einheit)
13      * @return Umfang des Rechtecks (selbe Einheit)
14      */
15      public static double umfangREck(double breite,double hoehe){
16          return 2 * (breite + hoehe);
17      }
18  }
```

Dokumentationskommentare: Extraktion

Erzeugung von Handbuch als HTML-Dateien in Verzeichnis doc:

```
$ javadoc -d doc GeometrieKommentiert.java
```

Ansehen: Datei `doc/index.html` mit Webbrowser öffnen

weitere Möglichkeiten: siehe Dokumentation zu `javadoc`

Übung: Javadoc

Aufgabenblatt 04, Aufgabe 09

Code

Lexikalische Ebene: Zeichen

Aufbau des Quelltextes aus Zeichen: **lexikalische** Struktur

Java-eigene Codeelemente bestehen aus sog. **ASCII-Zeichen** (i.W. Zeichen der US-Tastatur)

Sonderzeichen (ü ß å α € ...) in Java prinzipiell möglich in:

- Kommentar
- Zeichenkette
- Bezeichner

Hinweise:

- Sonderzeichen erfordern konsistente **Codierung** der Dateien (→ **UTF-8** verwenden), sonst **Übertragungsprobleme**
- **Sonderzeichen** darum besser generell **vermeiden**

Lexikalische Ebene: Token

Zeichen bilden

- White Space
- Kommentare

sowie **Token** („**Wörter**“) folgender Arten:

- **Schlüsselwörter**: `public class int ...`
- **Separatoren**: `{ } ; ...`
- **Operatoren**: `= + += ? : ...`
- **Literale**: `17 2.0 "Hallo" false ...`
- **Bezeichner**: `Kontakt lesen sc x ...`

(Namen für Klassen, Methoden, Parameter, Variablen etc.)

Syntaktische Ebene

Codeelemente dürfen **nicht beliebig** kombiniert werden

Aufbau des Codes aus Token: **syntaktische** Struktur

Fehler auf **lexikalischer** Ebene:

```
boolean 2b0rNot2b? = 2b | !2b; // falsch
```

Fehler auf **syntaktischer** Ebene:

```
public void static main(String[] args)
```

„gültiger“ Aufbau allgemein: **Syntax** der (Programmier-)Sprache
(bezieht lexikalische Ebene mit ein)

Syntax: Festlegung

wie Festlegung der Syntax?

- Beispiele anschaulich, aber:
ungenau, irreführend, spezifisch, ineffizient
- insbesondere zur automatischen Übersetzung benötigt:
präzise, eindeutig, allgemeingültig, minimal
- erfordert **formale** Beschreibung

wie beschreibt man formal z.B. eine if(-else)-Anweisung?

Formale Grammatik

Formale Grammatik: formale Festlegung der Syntax einer Sprache

es gibt verschiedene (aber ähnliche) Notationen für Grammatiken
im Folgenden Notation nach **Java-Sprachspezifikation** (→ Moodle)

Beispiel (vereinfacht):

Fallunterscheidung:

```
if ( LogikAusdruck ) { Anweisungen }  
if ( LogikAusdruck ) { Anweisungen } else { Anweisungen }
```

Leseweise: eine *Fallunterscheidung* ist

- *entweder if*, gefolgt von (, *logischem Ausdruck*,), ...
- *oder if*, gefolgt von ... sowie *else*, gefolgt von ...

Formale Grammatik: Notation

Fallunterscheidung:

```
if ( LogikAusdruck ) { Anweisungen }  
if ( LogikAusdruck ) { Anweisungen } else { Anweisungen }
```

Notation mittels folgender Elemente:

- Platzhalter (**Nichtterminale**) in *Kursivschrift*
- konkrete Zeichen/Token (**Terminale**) in *Normalschrift*
- **Produktionsregeln** der Form

Nichtterminal:

...

geben an, wofür *Nichtterminal* stehen kann:

erlaubte **Ersetzungen/Produktionen** eingerückt auf je einer Zeile

Hinweis: Ersetzung auf einer Zeile hier o.k., da Leerzeichen beliebig
⇒ jedoch **kein Stilvorbild**: nur „Richtigkeit“, nicht „Lesbarkeit“

Formale Grammatik: Umfang

Nichtterminale und Terminale heissen **Symbole**

eine **formale Grammatik** ist gegeben durch:

- eine Menge von **Nichtterminalen**
- eine Menge von **Terminalen**
(Zeichen auf lexikalischer, Token auf syntaktischer Ebene)
- **Produktionsregeln** für jedes Nichtterminal mit beliebig vielen **Ersetzungen** (Folgen von Nichtterminalen und Terminalen)
- Festlegung eines Nichtterminals als **Startsymbol**

Formale Grammatik: Option

häufiger Fall: Teil einer Ersetzung ist **optional**

Fallunterscheidung:

```
if ( LogikAusdruck ) { Anweisungen }  
if ( LogikAusdruck ) { Anweisungen } else { Anweisungen }
```

kürzere Schreibweise als **Option** [X]: X kommt 0- oder 1-mal vor

Fallunterscheidung:

```
if ( LogikAusdruck ) { Anweisungen } [else { Anweisungen }]
```

(Unterstreichung zur Verdeutlichung, nicht in Java-Spezifikation)

Formale Grammatik: Wiederholung

Wie lassen sich beliebig viele *Anweisungen* darstellen?

Wiederholung $\{ \underline{X} \}$: X kommt bel. oft (0-mal oder öfter) vor:

Fallunterscheidung:

if (LogikAusdruck) $\{ \{ \underline{\text{Anweisung}} \} \}$ $\underline{[\text{else } \{ \{ \underline{\text{Anweisung}} \} \}]}$

Hinweis: alternative Deklaration mittels „Plural“-Nichtterminal:

Anweisungen:

$\underline{[\text{Anweisung} \text{ Anweisungen }]}$

entsprechend darstellbar z.B. beliebig viele

- Methoden in einer Klasse
- Parameter in einer Parameterliste – aber Komma als Trenner:

ParamListe:

$\underline{[\text{Param } \{ , \text{Param } \}]}$

Formale Grammatik: Leerzeichen

Leerzeichen in Ersetzungen trennen Nicht-/Terminale voneinander

- in lexikalischen Regeln stehen sie nicht für White Space
- in syntaktischen Regeln für beliebig viele oder min. 1 WS:
 - beliebig viele (auch 0), wo Terminale unterscheidbar:

```
if(x<0){r=-x;}else{r=x;}
```

- min. 1, wo nicht:

```
int_x;
```

Bezeichner: Lexikalische Struktur

Wiederholung:

lexikalische Struktur von Bezeichner:

- erst ein (Gross-/Klein-)Buchstabe
- danach beliebig viele (Gross-/Klein-)Buchstaben/Ziffern

Eingabe BsBingo name1 r2d2 umwandlung24h i

Ergänzung:

für Buchstabe auch **Unterstrich** `_` verwendbar (aber **unüblich**!)

Anmerkung: **Schlüsselwörter** nicht als Bezeichner erlaubt

Bezeichner: (Lexikalische) Grammatik

Bezeichner:

BuchstabeUS { BuchstabeUSZiffer }

BuchstabeUSZiffer:

BuchstabeUS

Ziffer

BuchstabeUS:

Buchstabe

-

Buchstabe:

Grossbuchstabe

Kleinbuchstabe

Ziffer:

eines von 0 1 2 3 4 5 6 7 8 9

entsprechend Grossbuchstabe und Kleinbuchstabe

Formale Grammatik: erzeugte Sprache

eine **Ableitung** eines Nichtterminals entsteht

- durch Ersetzung des NT gemäß der entsprechenden Regel
- durch weitere Ersetzung von NT in der entstandenen Folge, gemäß den entsprechenden Regeln

dies lässt sich fortsetzen, bis die Ableitung **nur noch Terminale** enthält

von **formaler Grammatik erzeugte formale Sprache**:

Menge aller Folgen von Terminalen („Texte“), die sich aus dem **Startsymbol** in **endlich** vielen Schritten **ableiten** lassen

Anmerkung: in Java-Sprachspezif. (Text ist Inhalt einer Datei):

- *Input* als lexikalisches Startsymbol
- *CompilationUnit* als syntaktisches Startsymbol

Lexikalische Ableitung: Beispiel Bezeichner

mögliche **Ableitungen** für *Bezeichner*:

BuchstabeUS

BuchstabeUS BuchstabeUSZiffer

BuchstabeUS BuchstabeUSZiffer BuchstabeUSZiffer

...

aus *Bezeichner* als Startsymbol **erzeugte Sprache**:

A B C ... Y Z a b c ... y z _

AA AB ... AY AZ Aa Ab ... Ay Az A0 A1 ... A9 A_

...

_A _B ... _Y _Z _a _b ... _y _z _0 _1 ... _9 __

AAA AAB ... AAY AAZ AAa AAb ... AAy AAz AA0 AA1 ... AA9 AA_

...

__A __B ... __Y __Z __a __b ... __y __z __0 __1 ... __9 ___

...

Syntaktische Ableitung: Beispiel 1

Bestellung:

Ich moechte *Gericht* mit *Beilage* essen.

Gericht:

Ratatouille

Forelle

Steak

Beilage:

Reis

Kartoffeln

mit *Bestellung* als Startsymbol **erzeugte Sprache**
besteht aus 6 verschiedenen endlichen Ableitungen:

Ich moechte Forelle mit Reis essen.

Ich moechte Ratatouille mit Kartoffeln essen.

...

Syntaktische Ableitung: Beispiel 2

Satz:

Subjekt Praedikat [Objekt] .

Subjekt:

Mein Leguan

Der Torwart

Praedikat:

hasst

sucht

Objekt:

den Toaster

Fliegen

mit Satz als Startsymbol **erzeugte Sprache:**

Der Torwart sucht.

Mein Leguan hasst den Toaster.

Übung: aus Grammatik erzeugte Sprache

Aufgabenblatt 05, Aufgabe 01

Parsen

Parsen = (**lexikalische** und) **syntaktische Analyse**

i.W. **Umkehrung der Ableitung**:

Bestimmen und **Prüfen** der (lexikalischen und) syntaktischen Struktur eines gegebenen Codetextes, d.h.:

- **lässt sich** gegebener **Code aus Grammatik ableiten**?
- **wie** lässt sich gegebener Code aus Grammatik ableiten?
- welche Codeabschnitte entsprechen welchen Nichtterminalen?

wichtig:

Grammatik muss so angelegt werden, dass **Ableitungen eindeutig** sind (denn syntaktische Struktur liegt der Bedeutung des Codes zugrunde)

Übung: Grammatik ableiten

Aufgabenblatt 05, Aufgabe 02

Übung: Grammatik erstellen

Aufgabenblatt 05, Aufgabe 03

Syntax vs. Semantik

Syntaktisch korrekter Code ist nicht unbedingt korrekt:

```
1 public int beispiel(String s) {  
2     return "Hallo " + z + "!"; // 'z' nicht bekannt  
3                                 // Rückgabe String statt int  
4     int z = s;                 // Anweisung nach 'return'  
5                                 // Zuweisung String -> int  
6 }
```

(richtig: Zeilen 2 und 4 tauschen, int durch String ersetzen)

Semantik

„gültige“ Bedeutung: **Semantik** der (Programmier-)Sprache

Festlegung der Semantik:

meist **formlos oder semiformal** in **Sprachspezifikation**:

Wenn LogikAusdruck wahr ist, werden die Anweisungen im Block nach if ausgeführt, sonst die im Block nach else (falls vorhanden).

Hinweis: Semantik auch **formal** ($\rightarrow$ prüfbar) beschreibbar – wichtig für **kritische** Anwendungen, aber Spezifikation viel abstrakter $\rightarrow$ für „normalen“ Gebrauch zu komplex (und eingeschränkt)

Stil

was passiert bei „hässlichem“ Code?

- Übersetzer (Syntax und Semantik): **kein Problem**, wenn korrekt
- Entwicklerethos, Kollegen, Vorgesetzte, Kunden: **inakzeptabel**

Style Guidelines (**Stilrichtlinien**) schränken Regeln weiter ein:

- Schreibweisen für Bezeichner
- Einrückungen und Leerzeichen
- Umbruch auf mehrere Zeilen
- Reihenfolgen
- Verwendung von Sprachmitteln
- ...

Style Guidelines nicht „offiziell“, sondern von Organisationen/Firmen (z.B. Oracle, Google) → siehe **Dokumentation** in Moodle

Stil: bereits bekannte Regeln

- moderate **Zeilenlänge** (z.B. max. 100 Zeichen)
- **Leerzeilen** sparsam einsetzen
- Kommentare **kurz, aussagekräftig, präzise** halten
- **Klassenname** beginnt mit **Großbuchstaben**, dann „CamelCase“
- **Methoden- & Variablennamen** beginnen mit **Kleinbuchstaben**
- **Leerzeichen** hinter **if/while**, nicht vor **Methodenklammern**
- **öffnende Blockklammer** auf **selber Zeile**, danach **Zeilenumbruch**
- **Inhalt von Block** je um 2 Leerzeichen **einrücken**
- **schließende Blockklammer** auf **neuer Zeile**, dann **Zeilenumbruch** (sofern Anweisung nicht fortfährt, z.B. mit **else**)
- **Zeilenumbruch** nach jeder **Anweisung**
- **Leerzeichen** um binäre (und ternäre) **Operatoren**
- **Variablen** für **mehrfach/später verw.** Werte & „**Magic Numbers**“
- aussagekräftige **öffentliche** (**public**) **Namen** wählen
- kurze **interne** (Parameter, lokale Variablen-) **Namen** wählen

Übung: Style Guidelines

Aufgabenblatt 05, Aufgabe 04